Depth First Search (DFS)

Aim:

To traverse a graph using DFS.

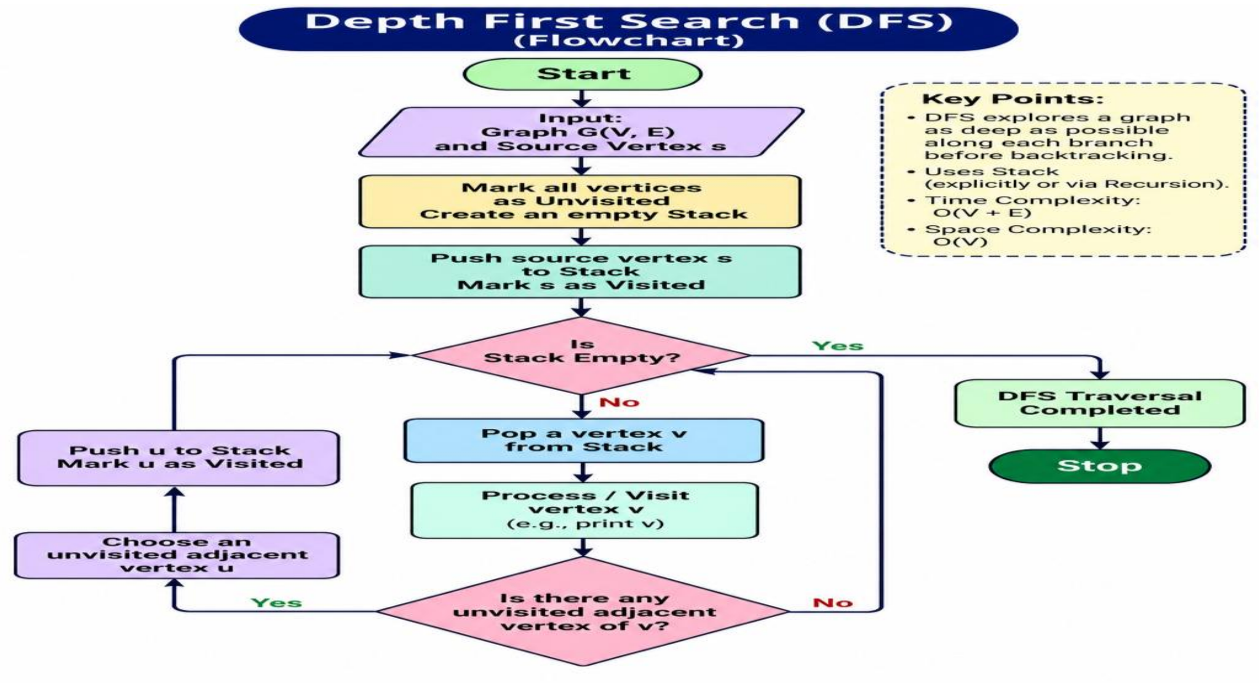
Objective:

To explore nodes deeply before backtracking.

Algorithm:

1. Start at root node.
2. Visit node.
3. Recursively visit adjacent nodes.
4. Backtrack when required.

FLOW CHART:



PYTHON CODE:

```
graph = {}


n = int(input("Enter number of nodes: "))
```

```
for i in range(n):
    node = input("Enter node name: ")
    neighbors = input(f"Enter neighbors of {node} (space-separated): ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")

visited = []

def dfs(node):
    if node not in visited:
        visited.append(node)
        for neighbor in graph[node]:
            dfs(neighbor)

dfs(start)

print("DFS Traversal:", visited)
```

OUTPUT:

```
IDLE Shell 3.13.14
File   Edit   Shell   Debug   Options   Window   Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========================================================================= RESTART: C:/Users/Sheik Mohamed/DFS.py ====
Enter number of nodes: 5
Enter node name: A
Enter neighbors of A (space-separated): B C
Enter node name: B
Enter neighbors of B (space-separated): D
Enter node name: C
Enter neighbors of C (space-separated): E
Enter node name: D
Enter neighbors of D (space-separated):
Enter node name: E
Enter neighbors of E (space-separated):
Enter starting node: A
DFS Traversal: ['A', 'B', 'D', 'C', 'E']
```

Result:

Graph traversed successfully.

Conclusion:

DFS explores depth before breadth.